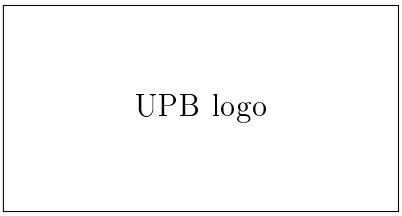
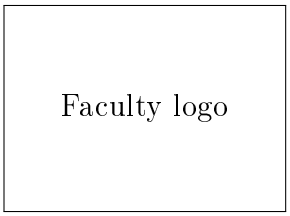
UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND
COMPUTERS
COMPUTER SCIENCE AND ENGINEERING
DEPARTMENT





DIPLOMA PROJECT

Scheduling de Task-uri În Sisteme Distribuite
TaskScheduler: workflow-aware scheduling for heterogeneous Kubernetes clusters

Bogdan Dumitrescu

Thesis advisor:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

Abstract

Scheduling a directed acyclic graph (DAG) on a heterogeneous cluster couples several decisions: which ready task to dispatch, where to run it, whether parent data should move, and whether urgent work justifies interrupting running work. Optimising one response-time objective can therefore worsen another, particularly when execution estimates are learned online and preemption discards progress.

This thesis presents *TaskScheduler*, an experimental workflow scheduling platform with two complementary implementations. The first is a deterministic discrete-event simulator with calibrated CPU, memory/cache, storage, network, thermal, energy, failure/repair, checkpoint, gang, release-time, and preemption models. It evaluates FCFS, Random, EFT, HEFT, historical simulator Legacy, exact Legacy v1, tuned exact Legacy, Adaptive, and Adaptive without preemption. The second is a Kubernetes-native control plane built from Workflow and TaskTemplate custom resources, a DAG controller, a custom Pod scheduler, persistent online profiles, a producer-local HTTP data plane, and node telemetry DaemonSets.

The simulation study covers five cluster families, fifteen workflow families, four overlapping-workload traces, and disjoint exploratory and confirmatory seeds. It retains 17,400 exploratory and 33,250 fresh-seed confirmatory matrix records. HEFT has the lowest confirmatory closed-DAG mean regret (1.69%) and P95 regret (5.75%); EFT is the close second. Under overlap, historical Legacy has the highest deadline hit rate (60.00%), Adaptive has the lowest P95 response regret (35.07%), and EFT/HEFT have the lowest median energy. A normalized 256-candidate search improves exact Legacy v1 on disjoint holdout and confirmation, but tuned Legacy does not become the global winner.

The live study executes 27 cyclically ordered trials across balanced, homogeneous, and resource-skewed logical `kind` profiles on one PC. All 135 workflows and 540 final task records succeed without task failure, container restart, collection error, or scheduler error. Tuned Legacy reduces aggregate v1 makespan by 3.50% and is lower in all three profiles. Relative to tuned Legacy, Adaptive reduces median workflow response by 19.40% but increases makespan by 14.80%, slightly worsens the P95, and performs 44 kill-and-restart preemptions.

The central result is consequently objective-specific rather than a universal win-

ner: HEFT/EFT are suitable static closed-DAG baselines, historical simulator Legacy leads modelled deadline success, tuned Legacy is preferred when the exact live scorer and batch completion are primary, and Adaptive is preferable when median regular-workflow response justifies restart cost. The `kind` measurements establish integration behaviour on one physical host; physical multi-machine validation remains future work.

Keywords: Kubernetes, workflow scheduling, heterogeneous clusters, thermal-aware scheduling, bandwidth-aware scheduling, UCB, ECT, CRDs, discrete-event simulation, kind.

Contents

Abstract **1**

1 Introduction **5**
1.1 Problem context 5
1.2 Research questions 5
1.3 Objectives 6
1.4 Contributions 7
1.5 Thesis structure 7

2 State of the art **8**
2.1 Heterogeneous DAG scheduling 8
2.2 Execution and interference models 8
2.3 Locality, fairness, and cluster schedulers 9
2.4 Online learning and exploration 9
2.5 Thermal awareness and preemption 10
2.6 Kubernetes extension mechanisms 10
2.7 Positioning of this work 11

3 Architecture and implementation **12**
3.1 Problem abstraction 12
3.2 Two execution surfaces 13
3.3 Discrete-event scheduler 13
3.4 Kubernetes control plane 14
3.5 Live ordering and ECT 15
3.6 Live signals 16
3.7 Producer-local data plane 17

4 Experimental methodology **18**
4.1 Implementation as the foundation 18
4.2 Experimental topology 18
4.3 Legacy weight search and factor checks 19
4.4 Simulation corpora 20

4.5 Live workload and protocol . . . 21
4.6 Derived comparisons . . . 21
4.7 Reproducibility and evidence artifacts . . . 22
4.8 Threats to validity . . . 23

5 Contributions **24**
5.1 Scheduling and simulation contributions . . . 24
5.2 Kubernetes engineering contributions . . . 24
5.3 Empirical contributions . . . 25
5.4 Explicit non-contributions . . . 25

6 Evaluation metrics **26**
6.1 Closed-workflow simulation metrics . . . 26
6.2 Overlapping-workload metrics . . . 26
6.3 Live Kubernetes metrics . . . 27
6.4 Aggregation and reporting . . . 27

7 Results **29**
7.1 Legacy weight optimization . . . 29
7.2 Closed heterogeneous DAGs . . . 30
7.3 Overlapping workflow arrivals . . . 32
7.4 Same-host Kubernetes results . . . 33
7.5 Cross-surface interpretation . . . 35
7.6 Answers to the research questions . . . 35

8 Conclusion and further work **37**
8.1 Summary . . . 37
8.2 Policy recommendation . . . 38
8.3 Limitations . . . 40
8.4 Further work . . . 40
8.5 Closing remarks . . . 41

Chapter 1

Introduction

1.1 Problem context

A workflow is represented as a DAG $G = (V, E)$ whose vertices are tasks and whose edges express execution or data dependencies. A task can run only after its predecessors complete. Once ready, it still has to be assigned to a node with compatible architecture and enough CPU and memory. Its completion time depends on the selected node, co-running tasks, image/startup delay, data location, network contention, temperature, and failure history. This is a heterogeneous DAG scheduling problem, for which even static formulations are generally computationally difficult; production systems therefore rely on heuristics such as HEFT [1].

Kubernetes exposes an appropriate execution substrate. Its scheduler filters infeasible nodes, scores feasible nodes, and binds a Pod to a selected node [16]. The default scheduler is general-purpose and intentionally does not know TaskScheduler's workflow graph, learned per-template runtimes, producer-local output locations, or experiment-specific risk model. Kubernetes also permits a separate scheduler to claim Pods through `schedulerName`, so domain-specific placement can be added without modifying Kubernetes.

The practical challenge is not simply adding more terms to a score. A more aggressive policy may reduce the response of an urgent workflow by preempting batch tasks, yet increase total completion time because those tasks restart. Likewise, a locality preference can avoid a data transfer but overload the producer node. The evaluation must therefore report multiple objectives and avoid declaring a universal winner from one aggregate metric.

1.2 Research questions

The work is organised around five research questions:

RQ1 Across controlled heterogeneous clusters and DAG families, which deployable heuristic gives the strongest closed-workflow makespan and regret behaviour?

RQ2 Under overlapping arrivals, how do deadline success, workflow response, total makespan, energy, and preemption trade off?

RQ3 Does seeded, out-of-sample weight optimization improve the exact Legacy scorer over its original vector, and which factors materially affect placement?

RQ4 Does the Kubernetes implementation execute the intended controller, placement, learning, locality, telemetry, and preemption paths under repeatable workloads and capacity profiles?

RQ5 Which conclusions agree across simulation and the live system, and which remain specific to one implementation or environment?

1.3 Objectives

The corresponding engineering objectives are:

O1. define validated schemas for heterogeneous clusters, DAGs, release times, priorities, gangs, checkpoints, and workload traces;

O2. implement deterministic resource and event models whose assumptions are explicit and independently testable;

O3. compare FCFS, Random, EFT, HEFT, three explicitly distinct Legacy variants, Adaptive, and an Adaptive no-preemption ablation over broad exploratory and untouched confirmatory matrices;

O4. parameterize the exact eight-factor Legacy scorer, tune it on disjoint training/validation cases, evaluate untouched holdout cases, and test factor ablations and controlled interventions;

O5. implement Workflow and TaskTemplate CRDs, a lifecycle controller, a custom scheduler, profile persistence, producer-local data movement, and node telemetry;

O6. execute cyclically balanced v1/tuned/Adaptive trials on three same-host `kind` capacity profiles and retain a machine-readable record of every workflow, task, placement, transfer, preemption, and scheduler error; and

O7. classify every conclusion by evidence source and state the physical-cluster validation that is still missing.

1.4 Contributions

The project contributes five concrete artifacts and findings. First, it provides a deterministic simulator that combines resource interference, network sharing, thermal dynamics, failures, checkpointing, gang scheduling, releases, and priority preemption in a single event model. Second, it provides a Kubernetes-native execution path with declarative workflow state and a separately deployable scheduler. Third, it makes the live Legacy equation reproducible in simulation, performs a normalized train/validation/holdout weight search, and checks every factor by ablation or controlled activation. Fourth, it contributes exploratory and fresh-seed confirmatory corpora plus cyclic three-policy evidence across three logical `kind` topologies. Fifth, it derives objective- and workflow-specific policy recommendations while identifying where simulation and same-host live execution support different scopes of claim.

This is an experimental systems contribution, not a proof that the Adaptive heuristic is globally optimal. The simulator and live system are separate implementations, the local thermal annotations are synthetic, and the completed Kubernetes experiment uses one physical host. These boundaries are part of the result rather than caveats to be hidden.

1.5 Thesis structure

Chapter 2 positions the work against DAG and cluster scheduling literature. Chapter 3 defines the model and implementation. Chapter 4 describes the evidence design. Chapter 6 defines the reported metrics, and Chapter 7 presents simulation and `kind` results. The final chapter gives objective-specific recommendations and the remaining physical-validation protocol.

Chapter 2

State of the art

2.1 Heterogeneous DAG scheduling

Let $\bar{w}_i$ be the average execution cost of task i over compatible processors and let $\bar{c}_{ij}$ be the average communication cost of edge (i, j). HEFT assigns each task the upward rank

$$\mathrm{rank}_u(i) = \bar{w}_i + \max_{j \in \mathrm{succ}(i)} \left(\bar{c}_{ij} + \mathrm{rank}_u(j) \right), \tag{2.1}$$

with the maximum equal to zero for a sink. Tasks are considered in non-increasing rank and placed on the processor that gives the earliest finish time [1]. The method is attractive because it is inexpensive, communication-aware, and directly represents the remaining critical path. It is therefore both a baseline in this work and the source of the simulator's task-ordering rule.

HEFT is fundamentally a static list heuristic: it assumes a known DAG and estimated computation/communication matrices. TaskScheduler adds release times, concurrent workflows, resource contention, failures, thermal state, gangs, and preemption. These additions do not preserve HEFT's original semantics automatically. In particular, the live controller exposes only ready Pods to the custom scheduler; the current live scheduler does not reconstruct all future CRD edges. Consequently, the simulator exercises full upward ranking, whereas the live Adaptive trial primarily exercises priority, ECT, profile learning, locality, risk terms, and preemption. This distinction prevents the common but incorrect claim that both surfaces run "the same HEFT policy."

2.2 Execution and interference models

Two classical performance models motivate the simulator's calibrated resource equations. Amdahl's law bounds parallel speedup by the serial fraction of a task [2]; TaskScheduler uses this relation when converting requested cores into effective throughput. The Roofline model bounds performance by the most restrictive computation or

bandwidth ceiling [3]. Analogously, the simulator estimates isolated task time as the maximum of CPU, memory, storage-read, and storage-write times rather than adding mutually overlapped phases.

The implemented model extends those abstractions with shared-resource allocation. CPU demand is capped by calibrated node throughput and scaled for SMT and thermal throttling. DRAM bandwidth and shared cache are divided among memory consumers. Storage rates use max-min allocation subject to sequential/random device limits. Routed network flows use progressive max-min filling over all links in their paths and a TCP window ceiling. These are explicit modelling choices, not claims that every real application overlaps resources perfectly or that a single fluid allocator captures kernel-level scheduling.

2.3 Locality, fairness, and cluster schedulers

Data locality and fairness can conflict. Quincy models scheduling as a flow problem to combine fair sharing with placement near input data [4]. Dominant Resource Fairness generalises max-min fairness to users with heterogeneous multi-resource demands [5]. TaskScheduler does not reproduce either algorithm. It uses producer-node metadata and estimated bytes to price remote dependencies, while workflow priority and virtual runtime order ready work. DRF remains useful context because CPU and memory feasibility alone do not define fairness when task shapes differ.

Large production schedulers emphasise scalable state management as much as the placement formula. Omega uses shared-state optimistic concurrency to support multiple schedulers [6]; Borg combines admission, priority, resource reclamation, and long-lived cluster management [7]. Sparrow demonstrates a decentralised low-latency design based on sampling rather than global optimisation [8]. TaskScheduler is much smaller: one custom scheduler maintains an in-memory model and binds ready Pods. The comparison is therefore architectural, not a claim of production-scale throughput.

2.4 Online learning and exploration

When runtime distributions are not known in advance, repeatedly selecting the currently best observed node can lock a scheduler into an early noisy estimate. Upper Confidence Bound (UCB) policies address the exploration/exploitation tradeoff by adding an optimism term that shrinks with the number of samples [14]. In a reward maximisation convention, a representative form is

$$\mathrm{UCB}_i(t) = \bar{x}_i(t) + \beta\sqrt{\frac{2\ln t}{n_i(t)}}, \tag{2.2}$$

where $n_i(t)$ is the number of observations for arm i.

The live implementation uses costs rather than rewards. It first forces coverage until every feasible node has at least three samples for a task template. It then adjusts ECT using the optimistic side of the profile estimate and continues learning from successful Pod completions. Legacy also forces coverage and retains a 10% random exploration branch after coverage. The simulation Adaptive policy does not learn UCB profiles during a run; it has direct access to scenario parameters and adds fixed energy, transfer, thermal, and failure terms. The shared name indicates a design family, not identical learning semantics.

2.5 Thermal awareness and preemption

Temperature-aware placement can reduce hot spots and cooling demand. Moore et al. showed that software placement informed by data-centre thermal behaviour could substantially reduce cooling cost in their modelled environment [9]. TaskScheduler's simulator uses a first-order resistance/capacitance model and a configured throttling curve. The live policy instead consumes a node temperature annotation and adds a fixed penalty when thermal headroom falls below 10 °C. The node agent first attempts Linux sysfs; when that sensor is unavailable in the local container environment, it maps load average to a synthetic value bounded between 40 and 85 °C. Retained same-host annotations therefore validate signal plumbing and score consumption, but cannot support a claim about real cooling or thermal throttling.

Kubernetes priority preemption removes lower-priority Pods when doing so makes a pending higher-priority Pod schedulable [10]. Victims may receive a graceful termination period, and discarded work creates a delay before capacity is usable. TaskScheduler's live planner applies stricter local budgets but immediately deletes each victim and later restarts it; it does not migrate memory state or checkpoint live Pods. The simulator separately supports checkpointable tasks and remaining-work restoration. The two mechanisms must not be conflated when interpreting results.

2.6 Kubernetes extension mechanisms

Custom resources extend the Kubernetes API, and pairing a custom resource with a controller creates a declarative reconciliation API [11]. TaskScheduler follows this pattern with Workflow and TaskTemplate CRDs. A controller creates Pods only when dependencies are satisfied and writes lifecycle state to the Workflow status subresource. A separate scheduler watches unscheduled Pods whose `schedulerName` is `ts-scheduler`, computes a placement, and calls the binding subresource. Node-local fileservers, bandwidth probes, and thermal collectors are DaemonSets, the standard mechanism for one

eligible Pod per node [12].

The live integration environment uses `kind`, which runs Kubernetes nodes as containers on a local container runtime [13]. This makes cluster creation and image loading repeatable, but all worker containers ultimately share one physical machine. The thesis therefore treats `kind` as control-plane and same-host systems evidence, not as a physical distributed-cluster benchmark.

2.7 Positioning of this work

The project does not propose a new optimality theorem, a new bandit bound, or a production-scale scheduler architecture. Its contribution is a controlled comparison and a working vertical integration. It asks what happens when DAG ordering, resource estimates, profile learning, data locality, thermal/failure risk, gangs, and bounded preemption meet in one testable system. The answer is empirical: the added mechanisms produce specialists and tradeoffs rather than one dominant policy.

Chapter 3

Architecture and implementation

TaskScheduler has two implementations of the same scheduling problem: a deterministic discrete-event simulator and a Kubernetes control plane. They share validated domain models and policy names where the intent is related, but they do not share a policy engine. This chapter defines the common abstraction and then describes each implementation separately.

3.1 Problem abstraction

A cluster is a graph $C = (N, L)$. Each node $n \in N$ has a node type, logical and physical CPU capacity, memory/cache capacity, storage limits, an idle/active power model, a thermal model, and an optional failure process. Each directed link $\ell \in L$ has payload bandwidth, latency, and loss parameters.

A workflow is a directed acyclic graph $G = (V, E)$. Task $i \in V$ has a resource request, compatible node types, a workload class, an optional release time and deadline, and optional checkpoint or gang metadata. An edge $(i, j) \in E$ carries precedence and may carry d_{ij} bytes of data. A workload is a time-ordered collection of workflow submissions, allowing policies to be evaluated under overlap rather than only on a closed DAG.

A placement is feasible when compatibility, CPU, memory, dependency, release, and gang constraints hold. No single scalar objective is declared universally correct. The evaluation observes workflow response, workload makespan, deadline success, energy, transfer, failure, and preemption together. This is important because the experiments show that minimising urgent response and minimising total makespan can select different policies.

3.2 Two execution surfaces

Table 3.1 states the implementation boundary used throughout the thesis.

Property	Simulator	Kubernetes control plane
Policies	FCFS, Random, EFT, HEFT, historical Legacy, exact Legacy v1, tuned exact Legacy, Adaptive, and Adaptive without preemption	exact Legacy v1, tuned exact Legacy, and Adaptive
DAG knowledge	Complete workflow, including future tasks and all edges	Controller knows the DAG; scheduler sees the ready-Pod frontier and currently reconstructs no future edges
Runtime knowledge	Explicit scenario demand and calibrated resource equations	Persisted medians, cold-start defaults, and online observations
Contention	Fluid CPU, cache, memory, storage, and routed-network allocation	Real Pod/container contention on the host kernel
Thermal state	First-order RC evolution, throttling, and shutdown limits	Node annotation and a threshold score penalty
Interruption	Failure/recovery, checkpoints, remaining work, and priority preemption	Immediate Pod deletion followed by controller retry; no live checkpoint restoration
Primary evidence	Controlled comparative experiment	End-to-end integration and descriptive same-host performance

Table 3.1: Semantic boundary between the two implemented execution surfaces.

3.3 Discrete-event scheduler

The simulator advances to the next release, transfer completion, resource completion, checkpoint, failure/repair, or thermal event. Running tasks share CPU, memory bandwidth, cache, storage, and network capacity. Network flows use progressive max-min filling over every link in their route and a TCP-window ceiling. CPU throughput applies calibrated per-core rates, Amdahl scaling, SMT efficiency, and thermal throttling. Temperature follows a first-order resistance/capacitance model. Failures interrupt resident tasks; checkpointable work resumes from its latest saved resource state, while other work restarts.

EFT greedily chooses the feasible task/node placement with the smallest estimated finish time. HEFT adds the upward-rank ordering of Eq. 2.1. The historical simulator policy named `legacy` starts from EFT and adds node-type specialisation and

small fragmentation terms. It predates the exact scorer adapter and is retained only as an earlier-project baseline; it is not the live eight-factor equation. The `legacy-v1` and `legacy-tuned` simulator policies instead invoke that exact live equation over deterministic simulator observations. Simulator Adaptive uses HEFT ordering, places higher-priority ready tasks first, and minimises

$$S_{i,n} = \widehat{F}_{i,n} + 0.002\widehat{E}_{i,n} + 0.001\widehat{D}_{i,n} + 10P_{i,n}^{\text{therm}} + 100P_{i,n}^{\text{fail}}, \tag{3.1}$$

where $\widehat{F}$ is estimated finish time in seconds, $\widehat{E}$ is active energy in joules, and $\widehat{D}$ is remote transfer in MiB. Thermal pressure increases as headroom falls within 20 °C of the throttle point. Failure probability is the configured hourly rate scaled by estimated execution time and capped at one. The Adaptive-no-preemption policy uses exactly this ordering and score with priority preemption disabled, making it an explicit ablation.

When a high-priority ready task cannot fit, simulator Adaptive may interrupt lower-priority running work. Victim selection prefers a set that releases sufficient resources while accounting for whether a checkpoint exists. This is functional preemption: it changes event state and either restores checkpointed remaining work or restarts the victim.

3.4 Kubernetes control plane

The deployable system consists of two CRDs, two control-plane Deployments, and three node agents (Figure 3.1):

1. **Workflow and TaskTemplate CRDs.** A Workflow stores task references, DAG dependencies, priority, deadline, and status. Reusable TaskTemplates define images, commands, resource requests, compatibility, and output-size hints.

2. `ts-controller`. The controller creates a Pod only after all of its parents finish, annotates it with task and producer-location metadata, and sets its `schedulerName` to `ts-scheduler`. It records outputs and status, retries interrupted tasks, and recursively fails descendants that can no longer become ready.

3. `ts-scheduler`. The scheduler watches its Pending Pods, reconstructs model objects, invokes the selected live policy, applies any victim deletions, and binds placements. It also records startup/runtime/failure profiles and persists them.

4. **Node agents.** The `ts-fileserver` exposes node-local output files. `ts-bw-probe` measures peer HTTP throughput. `ts-thermal` publishes a temperature signal. The latter two write Node annotations consumed by the scheduler.

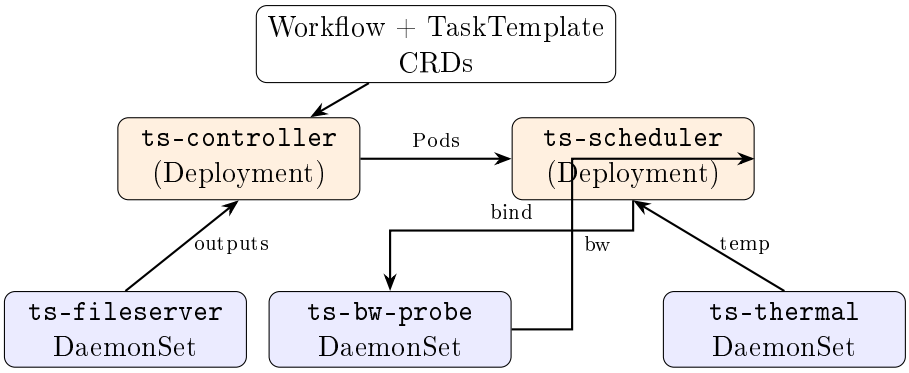


Figure 3.1: Deployable architecture. The controller owns DAG progression; the scheduler owns placement of the ready-Pod frontier.

3.5 Live ordering and ECT

The live policy interface returns explicit place, hold, and preempt actions. The static Kustomize manifest selects Adaptive; the `TS_POLICY` variable and experiment harness select explicit policy names for comparison. Exact Legacy applies an eight-factor score

$$L_{i,n} = \sum_{k=1}^{8} w_k \phi_k(i, n), \tag{3.2}$$

where the normalized features represent node-type affinity, resource fit, forecast availability, image warmth, load balance, observed failure rate, parent-data locality, and memory pressure. Higher scores are preferred. Both exact variants force three observations per compatible node and later retain a 10% random exploration probability; only their weights differ.

Factor	**Legacy v1**	**Legacy tuned**
Type affinity	30	23.6624
Resource fit	20	18.2647
Node availability	25	6.1406
Warm image	10	31.7960
Load balance	10	6.9166
Failure penalty	-10	-19.8013
Data locality	5	11.2579
Memory pressure	-15	-7.1604

Table 3.2: Exact Legacy weights. Both vectors have absolute weight scale 125; scalar rescaling would not change node ordering.

In simulation, `legacy-v1` denotes the original vector and `legacy-tuned` the selected vector in Table 3.2. In Kubernetes, `legacy` is a backward-compatible alias for `legacy-v1`. No result for the historical simulator `legacy` is treated as evidence about the exact live scorer.

Adaptive groups ready tasks by workflow and orders groups by descending effective priority, ascending virtual runtime, and arrival time. It then orders each group's ready tasks by stored upward rank. The current Kubernetes adapter, however, constructs scheduler-side workflow templates with an empty edge list. The controller enforces dependency readiness, but future successors are absent from the scheduler model; therefore the completed live experiment does not validate full HEFT upward-rank scheduling.

For candidate task i and node n, live Adaptive estimates

$$\widehat{F}_{i,n} = \max(t_{\text{now}}, o_n) + \widehat{s}_{i,n} + \sum_{p \in \text{parents}(i),\, a_p \neq n} \frac{d_{pi}}{b_{a_p,n}} + \widehat{r}_{i,n}, \tag{3.3}$$

where o_n is occupancy reserved earlier in the same scheduling tick, $\widehat{s}$ and $\widehat{r}$ are median startup and runtime profiles, a_p is the producer node, and b is measured or default bandwidth. Cold pairs use 2 s startup and 10 s runtime defaults. Same-node inputs have zero transfer cost.

Before exploiting this estimate, Adaptive schedules each feasible task-template/node pair until the least-observed candidate has three samples. It then applies an optimistic UCB cost adjustment and minimises

$$J_{i,n} = \widehat{F}_{i,n} + \min(0, U_{i,n} - \widehat{r}_{i,n}) + 50 q_{i,n} + 20 \mathbb{1}[h_n < 10], \tag{3.4}$$

where U is the profile-store UCB cost, q is observed failure rate, and h_n is thermal headroom in degrees Celsius. Gang placements are committed all-or-none within a scheduling tick. If no node can fit a non-gang task, the preemption planner may select strictly lower-priority victims subject to per-node and per-task anti-thrashing budgets. The live action is kill/restart: the scheduler deletes victims and the controller recreates interrupted tasks.

3.6 Live signals

Two signals are collected by DaemonSets:

Inter-node bandwidth.

`ts-bw-probe` on each node fetches a probe object from every peer's `ts-fileserver`, derives payload throughput, and writes per-peer bandwidth Node annotations consumed by the ECT estimator.

CPU temperature.

`ts-thermal` reads the available Linux thermal source and publishes

temperature and headroom annotations. The live score uses this as a threshold signal; unlike the simulator, it does not evolve an RC model.

3.7 Producer-local data plane

Each node owns a host-path directory `/var/lib/ts-data` that the controller mounts into every worker Pod at `/data/outputs`. Same-node parents are read through that mount with no network involvement. For cross-node parents, the controller injects `TS_PARENT_<NAME>_FILESERVER_URL` so the task script can `curl` the parent's output through the producer's `ts-fileserver`. This is intentionally simple: no shared filesystem, no object store — the data plane and the bandwidth signal share exactly the same path, so they stay consistent.

The controller propagates producer node and output-byte metadata to the child Pod. The scheduler converts that metadata into `input_bytes_by_node`, allowing Eq. 3.3 to account for remote bytes before binding. At execution time, same-node children read the host path directly; cross-node children use the producer's HTTP endpoint. Thus the locality score and observed transfer traverse the same producer identity, although estimated and realised bandwidth can still differ.

Chapter 4

Experimental methodology

4.1 Implementation as the foundation

A working implementation is treated as a prerequisite for any quantitative claim. Two execution surfaces are maintained in parallel, but they are not the same program and their outputs are not assumed to be numerically interchangeable:

1. A **discrete-event simulator** (`simulation/`) executes independent simulator policy implementations against explicit cluster, workflow, and workload schemas. Given the same configuration and seed, the event order and sampled uncertainty are reproducible. This surface supports controlled algorithm comparison, including failures, repair, checkpointing, thermal throttling, transfer contention, gangs, and priority preemption.

2. A **Kubernetes control plane** executes real Pods through the Workflow controller and custom scheduler. Its Legacy and Adaptive policies are live implementations over Kubernetes state, learned profiles, and node annotations. They share selected data models with the simulator, but not an identical scheduling code path.

The evidence is therefore tiered. Simulation results provide controlled comparative evidence about policy behaviour under the stated model. Same-host `kind` results establish control-plane correctness and provide descriptive live performance evidence. Physical multi-machine `k3s` validation is specified by repository scripts but was not performed for this evaluation. The thesis does not infer physical network, thermal, or failure behaviour from the `kind` trials.

4.2 Experimental topology

The live evaluation uses local `kind` clusters with one control-plane node and three workers. Each Kubernetes node is a Docker container on the same Windows work-

station. Advertised capacity is matched by Docker cgroup CPU and memory limits, while output storage is node-local. This creates meaningful logical capacity and locality differences without pretending that the workers are independent machines.

Topology	Worker	Type	CPU / memory	Advertised I/O
Balanced	`cpu-1`	CPU_OPT	2.00 / 1024 MiB	100
	`mem-1`	MEM_OPT	1.00 / 1536 MiB	100
	`io-1`	IO_OPT	1.00 / 768 MiB	1000
Homogeneous	`equal-1..3`	CPU_OPT	1.25 / 1280 MiB each	300
Skewed	`cpu-fast`	CPU_OPT	2.50 / 1280 MiB	100
	`mem-large`	MEM_OPT	0.75 / 1792 MiB	100
	`io-fast`	IO_OPT	0.75 / 768 MiB	1500

Table 4.1: Three recreated same-host `kind` capacity profiles. Each also has one control-plane container. Units in the final column are scheduler-relative capability hints, not measured physical IOPS.

Recreating the cluster for each profile avoids stale Kubernetes state and makes the intended capacity ratios explicit. The three profiles preserve real reconciliation, Pod creation, binding, preemption, retry, HTTP transfer, and profile-learning paths. They do not provide independent failure domains, physical links, or externally valid thermal measurements. Thermal annotations are interpreted only as verification that the scheduler consumes the signal.

4.3 Legacy weight search and factor checks

The Legacy study changes only the weights in Eq. 3.2. Candidate signs are fixed—rewards remain non-negative and penalties non-positive—and every vector is normalized so that

$$\sum_{k=1}^{8} |w_k| = 125. \tag{4.1}$$

This removes scalar duplicates because multiplying all terms by one positive constant cannot change a node ordering.

A seeded log-normal search generates 256 local-to-broad perturbations of Legacy v1, including the baseline itself. Ninety training cases cover five clusters, nine workflow families, and seeds 0–1. The best 25 training candidates plus the baseline advance to 45 validation cases formed from three different workflow families and seeds 17, 29, and 43. Selection minimizes the fixed objective

$$\begin{aligned} J_w = {} & 0.45\,\overline{r}_M + 0.20\,r_{M,95} + 0.15\,\overline{r}_T + 0.10\,\overline{r}_E \\ & + 0.05\,\overline{r}_D + 0.05\,r_L, \end{aligned} \tag{4.2}$$

where ratios compare candidate metrics with Legacy v1 for aligned cases. A candidate is eligible only if validation objective, deadline misses, and retries are no worse than v1 and its P95 makespan ratio is at most 1.05. The selected vector is then evaluated, without further selection, on 75 holdout cases using three unseen workflow families and seeds 101, 103, 107, 109, and 113. Both empty and independently prewarmed profile stores are reported; warm stores contain twelve seeded observations per compatible task/node pair.

Factor evidence uses a second fresh block. Seeds 300–309 cross all five clusters and fifteen workflows, producing 750 cases for Legacy v1, tuned Legacy, and each scale-renormalized leave-one-factor-out variant. Because broad corpora need not activate every term, separate two-node interventions test whether warm image, observed failure, and memory pressure can each reverse a placement while the other score inputs are controlled.

4.4 Simulation corpora

The closed-workflow matrix crosses five cluster families with fifteen workflow families. Cluster scenarios cover an asymmetric five-node edge network, a balanced heterogeneous four-node cluster, a homogeneous four-node cluster, a locally calibrated host model, and an unreliable thermal three-node cluster. Workflow scenarios cover cache-sensitive, checkpoint/recovery, CPU-heavy, deadline-critical, data-heavy, deep, diamond, fan-in, fan-out, gang, I/O-heavy, linear, memory-heavy, mixed, and wide DAGs.

The exploratory phase evaluates every cluster/workflow pair under seeds 0–19 with FCFS, Random, EFT, HEFT, historical Legacy, exact Legacy v1, tuned exact Legacy, Adaptive without preemption, and Adaptive. For every seed, declared CPU, memory, storage, network, and power uncertainty is sampled deterministically from a digest of cluster identifier and seed. The resulting corpus has

$$5 \text{ clusters} \times 15 \text{ workflows} \times 20 \text{ seeds} \times 9 \text{ policies} = 13{,}500 \tag{4.3}$$

deployable-policy records. Exhaustive static assignment is additionally evaluated where the assignment space is tractable, adding 300 reference records and bringing the saved exploratory total to 13,800.

The overlapping-workload matrix crosses the same five clusters with four arrival traces: data-locality wave, mixed arrivals, priority burst, and recovery/gang mix. Each trace contains multiple workflow submissions with explicit release offsets; selected submissions also have deadlines. With the same nine policies and twenty uncertainty seeds,

this corpus contains

$$5 \times 4 \times 20 \times 9 = 3{,}600 \tag{4.4}$$

records over 400 comparable cluster/trace/seed cases.

Confirmation uses untouched seeds 200–249 and the seven serious common policies: EFT, HEFT, all three Legacy variants, Adaptive without preemption, and Adaptive. FCFS and Random remain in exploration as controls but are not rerun after their broad inferiority is established. The closed confirmatory matrix therefore contains $5 \times 15 \times 50 \times 7 = 26{,}250$ rows over 3,750 cases; the overlapping matrix contains $5 \times 4 \times 50 \times 7 = 7{,}000$ rows over 1,000 cases. Exploratory and confirmatory summaries remain separate so the latter is not silently pooled into model selection.

4.5 Live workload and protocol

The final live harness compares exact Legacy v1, tuned exact Legacy, and Adaptive. Each trial first submits a six-task best-effort CPU pressure workflow. Seven seconds later it submits a non-preemptable two-task critical workflow, followed by a three-stage data pipeline, a one-to-four fan-out, and four independent heterogeneous tasks. Thus every trial has five workflows and twenty final task records.

Each policy is run three times on each topology. Cyclic order gives every policy each execution position once: v1/tuned/Adaptive, tuned/Adaptive/v1, and Adaptive/v1/tuned. Before each trial the harness deletes prior experiment objects, deploys the requested policy, and verifies policy identity from logs. A profile path is isolated by policy and topology run but shared across that policy's three repetitions, so later repetitions can use earlier observations without cross-policy contamination. Exact Legacy weights are copied into the artifact. The harness waits for all five workflows, records CRD, Pod, transfer, placement, preemption, and scheduler-log evidence, and pauses six seconds before the next trial. These are sequential same-machine repetitions with evolving profiles, not IID samples; live summaries are descriptive and have no population confidence interval. Per-repetition makespan and response are retained to test whether later profile states coincide with improvement, but cyclic execution position, image/cache state, and shared-host noise prevent attributing a trend to learning alone.

4.6 Derived comparisons

For a closed case $c = (\text{cluster}, \text{workflow}, \text{seed})$, policy regret is

$$R_{p,c} = \frac{M_{p,c}}{\min_{q \in Q_c} M_{q,c}} - 1, \tag{4.5}$$

where Q_c is the set of deployable policies common to that comparison. A policy "win" means $R_{p,c} \leq 10^{-9}$, so exact simulator ties count for every tied policy. Mean-regret intervals are percentile bootstrap intervals from 2,000 deterministic resamples (bootstrap seed 2026). The static oracle is excluded from this denominator and reported separately as a gap on the 300 exploratory cases where it exists. This makes exploratory and confirmatory regret directly comparable. Intervals describe variation across the designed corpus; they do not imply that scenarios are a random sample of all deployments.

An independent optimistic lower bound is the maximum of (i) a critical path formed from each task's fastest compatible isolated execution and (ii) aggregate CPU, memory-bandwidth, storage-read, and storage-write work divided by total corresponding cluster capacity. The bound ignores contention, communication, cold starts, failures, and placement compatibility interactions, so a ratio above one is expected. The static oracle enumerates node assignments but still executes them through the simulator; it is a small-case reference, not a global lower bound.

For overlapping case $c = (\text{cluster}, \text{trace}, \text{seed})$, the analogous regret uses mean per-submission response instead of makespan. Ranking sorts median response regret, then 95th-percentile response regret, then policy name for deterministic ties. Deadline hit rate, workload makespan, energy, and preemption count remain co-equal reported outcomes rather than being folded into that rank.

4.7 Reproducibility and evidence artifacts

All scenario inputs are versioned under `scenarios/`. The `run_experiment_lab.py` CLI selects scenario sets, policy sets, half-open seed ranges, uncertainty sampling, oracle limits, and output directories. It writes raw JSON Lines and CSV before deriving JSON and Markdown summaries. The all-version exploratory corpora are retained under `results/final-all-legacy-20seed/` and `results/final-all-workload-20seed/`. Fresh-seed rows are under `results/confirmatory-single-seeds200-249/` and `results/confirmatory-workload-seeds200-249/`. The deterministic `analyze_final_evidence.py` synthesis is generated from those raw JSON Lines files as `results/final-evidence-synthesis.json` and its Markdown companion. Weight-search, ablation, and controlled-intervention artifacts are retained separately so aggregate policy comparison cannot be confused with parameter selection.

The live harness is `run_k8s_experiment.py`; `scripts/run-kind-topology-matrix.ps1` recreates and validates all three profiles. One JSON document per topology retains cluster metadata, timestamps, exact weights, every workflow and final task record, scheduler events, and aggregate summaries under `results/k8s-kind-topologies/`. The earlier balanced Legacy/Adaptive pilot remains under `results/k8s-kind/` as formative evidence rather than being merged into the three-policy matrix. Physical `k3s`

scripts remain an unexecuted validation path and are not counted as experimental evidence.

4.8 Threats to validity

The simulator is a model, not a calibrated digital twin of every target. Its fluid sharing, overlap assumptions, uncertainty ranges, and scenario deadlines can influence policy order. The broad factorial matrix reduces dependence on one topology but does not remove model risk. Exact ties are common because several policies choose the same placement on simple ready frontiers. The selected Legacy vector is the best eligible validation candidate among 256 samples for one fixed objective; it is not a global optimum, and the holdout does not cover every possible workload distribution.

The live experiment executes real reconciliation, binding, deletion, retry, data movement, and containers, but all logical workers share one Windows/Docker host. Container caps create visible heterogeneity without creating independent hardware, network links, or thermal domains. Injected temperatures validate metadata consumption only. Three sequential repetitions per policy and topology can reveal repeatable mechanisms and directional agreement, but not estimate population-level performance. Profile evolution and shared-host background load remain possible order effects despite cyclic balancing. Physical multi-machine validation is therefore the principal external-validity gap.

Chapter 5

Contributions

5.1 Scheduling and simulation contributions

1. A schema-driven deterministic discrete-event simulator for heterogeneous DAG workloads. It combines explicit CPU, cache, memory, storage, routed-network, thermal, power, failure/repair, checkpoint, release-time, gang, and priority-preemption behaviour in one executable model.

2. Nine comparable simulator policies: FCFS and seeded Random controls; EFT and full-DAG HEFT baselines; historical simulator Legacy; exact Legacy v1 and tuned Legacy; and Adaptive with and without preemption. The Adaptive pair changes only interruption, while the exact Legacy pair changes only weights.

3. A workload compiler that preserves overlapping submission times and returns per-submission response, deadline, and lateness metrics from one shared simulation run.

4. Independent optimistic lower bounds and an exhaustive static assignment reference for tractable DAGs. These anchors distinguish relative policy wins from absolute distance to an idealised bound.

5. A seeded, scale-normalized Legacy weight search with disjoint training, validation, and holdout workflows; deterministic warm profiles; fresh-seed leave-one-factor-out evaluation; and controlled placement reversals for factors inactive in the broad corpus.

5.2 Kubernetes engineering contributions

1. A Kubernetes-native vertical path comprising Workflow and TaskTemplate CRDs, a DAG controller, a custom scheduler, profile persistence, RBAC, and fileserver, bandwidth, and thermal DaemonSets. Root Kustomize manifests provide one installation entry point.

2. A precise ownership split: the controller owns dependency progression and terminal-state propagation, while the scheduler owns ready-Pod ordering, placement, binding, learning, and victim selection. Interrupted Pods return through the controller's retry path; descendants of failed tasks terminate recursively rather than waiting forever.

3. A producer-local data contract that carries node identity and output byte counts from completed parents into scheduling and task execution. The same metadata drives ECT transfer cost, direct local reads, and remote HTTP fetches.

4. A live Adaptive policy that joins priority and virtual-runtime ordering with occupancy-aware ECT, coverage-first profile learning, an optimistic UCB adjustment, failure history, thermal headroom, gang rollback, and bounded kill/restart preemption.

5. A repeatable live experiment harness that cyclically balances three policies, isolates profiles by policy/topology, verifies scheduler identity and exact weights, cleans resources, and retains workflow, task, transfer, placement, preemption, restart, and error evidence. A matrix runner recreates balanced, homogeneous, and resource-skewed `kind` clusters.

5.3 Empirical contributions

The saved corpora provide four complementary findings. First, the 13,800 exploratory and 26,250 untouched-seed confirmatory closed-DAG records show that informed list schedulers have family-specific tails; HEFT has the lowest mean and P95 regret, while no policy dominates every cluster/workflow family. Second, 3,600 exploratory and 7,000 confirmatory overlapping-workload records expose deadline, response, energy, and makespan tradeoffs and isolate simulator preemption. Third, the Legacy search, broad ablation, and controlled factor interventions separate weight quality from factor activation. Finally, retained live `kind` trials exercise binding, persistent learning, producer-local data, and real kill/restart actions. This mechanism-level account is more useful than declaring a single winner.

5.4 Explicit non-contributions

This work does not prove scheduler or weight optimality, establish a new UCB regret bound, or claim production-scale scheduler throughput. Simulator and live policies are related implementations rather than shared code. The live trials do not exercise full-DAG HEFT ranking, real thermal throttling, independent network links, or physical node failures. Multi-architecture build and `k3s` deployment scripts exist, but successful multi-machine execution is not evidence produced by this evaluation.

Chapter 6

Evaluation metrics

Metrics are defined against the clocks and records of each execution surface. Simulator seconds are virtual time under a sampled scenario; live seconds are differences between UTC Kubernetes/controller timestamps. They are compared qualitatively across surfaces but are not pooled into one statistical sample.

6.1 Closed-workflow simulation metrics

For a single workflow released at zero, *makespan* $M_{p,c}$ is the virtual time at which all tasks reach a terminal state. The primary relative measures are regret $R_{p,c}$ from Eq. 4.5, win rate, and the ratio $M_{p,c}/L_c$ to the optimistic lower bound. Regret normalises large and small scenarios while the bound ratio preserves an absolute optimality reference independent of the policy set. On the tractable exploratory subset, a second table reports each deployable policy's makespan gap to exhaustive static assignment. The oracle is not mixed into the deployable regret denominator.

The closed record also retains total node energy in joules, routed network transfer in MiB, cold starts, task retries, node failures, maximum node temperature, deadline success, and positive lateness. Deadline hit rate is computed only over workflows with a configured deadline. A policy can therefore have good makespan regret but still be undesirable on energy, transfer, reliability, or thermal grounds.

6.2 Overlapping-workload metrics

For submission w with release time a_w and terminal time f_w, response is

$$T_w = f_w - a_w. \tag{6.1}$$

Each cluster/trace/policy/seed record stores mean and nearest-rank 95th-percentile submission response. Workload makespan is the virtual span from the first release to

the final terminal event. Response regret uses the minimum mean response among recorded policies for the same cluster, trace, and seed.

Deadline hit rate is the fraction of deadline-bearing submissions that finish by a_w+D_w. The record additionally stores deadline misses, task preemptions, retries, node failures, total energy, and network transfer. Adaptive's preemption count is interpreted together with its no-preemption ablation; count alone does not indicate whether an interruption was beneficial.

6.3 Live Kubernetes metrics

Live workflow response is the interval from Workflow-object creation to its recorded finish timestamp. A trial's makespan spans the earliest of its five creation timestamps to the latest finish. The critical-response metric applies the same definition to the delayed critical workflow. Per-topology policy summaries report the median over 15 workflow responses, their nearest-rank 95th percentile, the median of three trial makespans, and the median critical response over three critical workflows. Aggregate summaries apply the same definitions to all nine trials and 45 workflows for a policy rather than averaging topology medians.

Task evidence includes assigned node, final phase/state, startup, container and status runtimes, output bytes, measured transfer seconds, and local/remote transfer mode. Operational counters include successful workflows, final task records, scheduler binding lines, kill/restart preemption lines, container restart counts, failed task states, and scheduler error lines. These counters are essential validity checks: short response obtained by losing work would not be a successful result.

6.4 Aggregation and reporting

Medians and nearest-rank 95th percentiles are preferred for skewed regret and response distributions. Closed-policy tables additionally report mean regret and its deterministic 2,000-resample percentile interval because means expose severe tail costs hidden by the many exact ties. Percent differences in the live comparison use Legacy as the reference only when reproducing the earlier pilot. General pairwise comparisons use policy b as the declared baseline:

$$\Delta_{p/b} = 100\frac{x_p - x_b}{x_b}. \tag{6.2}$$

Negative values are improvements only for duration metrics. Energy, preemption, and completion are reported directly rather than forced into that convention.

Jain fairness, continuous thermal exposure, per-task isolated slowdown, and Pending-to-Scheduled latency were considered during design but are not present in the definitive

result artifacts. They are not used to support the conclusions below.

Chapter 7

Results

This chapter reports the three evidence tiers separately. The closed exploration contains 13,800 rows over 1,500 comparable cases, including 300 separate small-case oracle rows; fresh-seed confirmation contains 26,250 rows over 3,750 cases. Overlapping exploration contains 3,600 rows over 400 cases; confirmation contains 7,000 rows over 1,000 cases. The earlier live pilot contains six completed trials, 30 workflows, and 120 final task records. The final live tier recreates three logical capacity profiles on the same physical host and compares exact Legacy v1, tuned exact Legacy, and Adaptive with cyclic trial order.

7.1 Legacy weight optimization

The selected vector in Table 3.2 ranks first among the validation finalists with objective 0.95898 relative to Legacy v1. It has no validation deadline miss or retry and preserves a P95 makespan ratio of 1.0. This is a selection result within the declared 256-candidate search, not a claim that no better vector exists.

Table 7.1 reports the untouched 75-case holdout. Absolute columns are arithmetic means across cases. The objective is the weighted mean of aligned per-case ratios in Eq. 4.2, so it is not reconstructed by simply dividing the displayed aggregate means.

Profiles	Policy	Mean makespan (s)	P95 (s)	Mean task response (s)	Mean energy (J)	Mean transfer (MiB)	Retries
Cold	Legacy v1	49.5156	139.2561	39.0229	2027.3000	794.000	10
Cold	Legacy tuned	45.7993	139.2561	37.5804	1867.6894	229.867	5
Warm	Legacy v1	43.2603	139.2561	37.1900	1754.1554	434.667	5
Warm	Legacy tuned	42.8193	139.2561	36.9444	1738.0269	272.533	5

Table 7.1: Legacy v1 and tuned Legacy on the untouched holdout. Neither profile condition has a deadline miss. Cold-profile node failures are 15 for v1 and 10 for tuned; warm-profile failures are 10 for both.

The cold-profile tuned/v1 objective is 0.90022. Mean aligned case ratios improve by 11.94% for makespan, 15.67% for task response, 11.19% for energy, and 22.73% for transfer. Under independent warm profiles the objective is 0.97525; corresponding

improvements are 2.31%, 4.91%, 1.87%, and 10.25%. P95 makespan and deadline lateness are unchanged in both conditions. Tuning therefore improves central and transfer behaviour, especially before profiles are populated, but does not improve the observed worst-case makespan quantile.

The fresh-seed factor study in Table 7.2 gives a different kind of evidence. Every ablation is renormalized to absolute scale 125, so the result measures redistribution after removing one factor rather than merely shrinking the whole score.

Variant	**Objective vs v1**	**Change vs tuned**
Without availability	0.977168	−0.000111
Legacy tuned	0.977279	0.000000
Without warm image	0.977279	0.000000
Without failure penalty	0.977279	0.000000
Without memory pressure	0.977279	0.000000
Without resource fit	0.977392	+0.000114
Legacy v1	1.000000	+0.022721
Without load balance	1.050907	+0.073628
Without data locality	1.062057	+0.084779
Without type affinity	1.681783	+0.704505

Table 7.2: Leave-one-factor-out results over 750 fresh-seed cases per variant. Lower objective is better.

Type affinity, locality, and load balance are consequential over this corpus. Removing availability improves the objective by only 0.000111, while removing resource fit worsens it by a similarly small 0.000114. These are post-selection observations on a different corpus, not a reason to tune a ninth vector after seeing confirmation. In particular, the chosen vector is retained for the live test rather than optimizing against every later comparison.

Warm image, failure penalty, and memory pressure do not change the aggregate broad-corpus result, but controlled interventions show that all three implementations are active. With the full score, a cached image changes the choice from `cold` to `warm`; failure history changes `risky` to `healthy`; and low free memory changes `pressured` to `safe`. Setting only the named factor to zero reverses each choice. The broad tie therefore measures factor prevalence in these scenarios, while the controlled cases establish causal ability to affect placement.

7.2 Closed heterogeneous DAGs

Table 7.3 ranks deployable policies by median, 95th-percentile, and mean makespan regret. Since at least 60% of cases are exact wins or ties for every confirmatory policy, median regret is zero throughout and is omitted as non-discriminating. The table uses

only seeds 200–249, which were not used for Legacy weight selection.

Policy	Win rate	Mean regret (95% interval)	P95 regret	Deadline hit
HEFT	91.97%	1.69% [1.40, 1.98]	5.75%	83.60%
EFT	91.97%	1.85% [1.58, 2.15]	10.05%	83.60%
Historical Legacy	91.89%	2.50% [2.17, 2.84]	17.73%	83.60%
Legacy tuned	83.87%	6.97% [6.14, 7.79]	54.68%	83.60%
Adaptive	86.61%	4.96% [4.42, 5.53]	55.14%	83.60%
Adaptive no-preempt	86.61%	4.96% [4.42, 5.53]	55.14%	83.60%
Legacy v1	74.53%	13.07% [11.88, 14.17]	84.32%	83.60%

Table 7.3: Fresh-seed closed-workflow confirmation over 3,750 cluster/workflow/seed cases per policy. Intervals are deterministic percentile bootstrap intervals over case regrets, not live-run confidence bounds.

HEFT has both the lowest mean regret, 1.69%, and lowest P95 regret, 5.75%. EFT follows at 1.85% mean and 10.05% P95 regret. Their mean intervals overlap, so HEFT is the best central recommendation for this corpus rather than a claim of isolated superiority. Historical Legacy is third by mean regret at 2.50%. All seven policies have the same 83.60% deadline rate in confirmation, so that metric does not select a closed-DAG winner.

When exploration is recomputed over the same seven-policy set, every policy retains its ordinal position under seeds 200–249. The stable ranking across disjoint seed blocks strengthens internal replication; it does not make the synthetic scenario families a random population.

Tuned exact Legacy improves exact Legacy v1 without overtaking the three leading list policies. Across paired confirmatory cases, its mean makespan ratio to v1 is 0.9714 (95% bootstrap interval [0.9677, 0.9751]), with 550 wins, 3,150 ties, and 50 losses. Its P95 regret falls from 84.32% to 54.68%. This supports the selected weights for deployments that require the exact live scorer, but does not turn that scorer into the overall closed-DAG winner.

The Adaptive variants have identical aggregate closed-workflow results. Priority preemption provides no measured advantage in this corpus, where one DAG is released as a closed job. Their energy, transfer, thermal, and failure penalties instead change some placements relative to pure EFT/HEFT and slightly worsen the overall regret tail. This is a useful negative result: adding more score terms does not monotonically improve makespan.

The exploratory controls illustrate why median regret alone is insufficient. FCFS ties a recorded best in 61.33% of exploratory cases yet has 128.77% mean and 691.00% P95 regret. Random has 347.97% mean and 1673.04% P95 regret. The informed policies are therefore not merely benefiting from abundant capacity; placement information matters in the hard tail.

The exhaustive static assignment is excluded from deployable-policy regret and

reported separately on its 300 tractable exploratory cases. HEFT has a 3.06% mean gap to that reference (95% interval [2.22%, 3.95%]); EFT and historical Legacy each have a 3.85% mean gap. Median gap is zero for all informed policies, but HEFT's P95 gap is 29.55%. The oracle therefore anchors a small subset without changing the denominator of the main policy comparison.

Confirmatory family analysis rejects a universal specialist. HEFT is the sole mean-regret leader for cache-sensitive DAGs. Adaptive and its no-preemption ablation lead fan-out and mixed families and share the diamond lead with tuned Legacy. EFT and HEFT lead CPU-heavy DAGs; EFT, HEFT, and historical Legacy tie across fan-in, I/O-heavy, and memory-heavy families. Linear, deep, data-heavy, failure-recovery, and gang families contain still broader ties. Policy selection should therefore use workflow and topology knowledge rather than only the global rank.

7.3 Overlapping workflow arrivals

Table 7.4 reports response, deadline, makespan, energy, and interruption outcomes over 1,000 fresh-seed cluster/trace/seed cases per policy.

Policy	Median response regret	P95 regret	Deadline hit	Median makespan (s)	Median energy (J)	Mean preemptions
Adaptive	0.00%	35.07%	53.33%	55.070	3738.191	0.76
Historical Legacy	0.00%	74.82%	60.00%	62.421	3738.191	0.00
Adaptive no-preempt	0.26%	38.54%	53.33%	55.070	3738.191	0.00
EFT	0.27%	71.84%	53.33%	55.070	3080.432	0.00
HEFT	0.27%	71.84%	53.33%	55.070	3080.432	0.00
Legacy tuned	1.08%	54.70%	53.33%	62.421	3738.191	0.00
Legacy v1	3.26%	77.51%	46.67%	67.084	3828.224	0.00

Table 7.4: Fresh-seed overlapping-workload confirmation over 1,000 cases per policy. Deadline hit rate is averaged over deadline-bearing submissions.

Adaptive has the lowest response tail: 35.07% P95 response regret and zero median regret. Historical Legacy also has zero median regret and the highest deadline hit rate, 60.00%, but its 74.82% response tail and 62.421 s median makespan expose the cost. Adaptive, Adaptive no-preempt, EFT, and HEFT share the lowest median makespan, 55.070 s. Thus the ranking changes with the objective.

EFT and HEFT have the lowest median energy, 3080.432 J, 17.60% below the 3738.191 J of Legacy and both Adaptive variants. HEFT is weakly dominated by EFT in response because their reported medians match while EFT has lower arithmetic mean response regret. More generally, response tails, energy, deadline success, and makespan do not select one common policy.

Adaptive performs 0.755 preemptions per case on average; all other policies perform none. It shares median makespan, energy, and deadline rate with its no-preemption ablation while improving P95 response regret from 38.54% to 35.07%. Preemption therefore helps the normalised response tail but is not the cause of the broad 55.070 s

makespan result. A deployment concerned about restart cost can choose the no-preemption variant with a modest aggregate tail penalty in this model.

Tuned Legacy also consistently improves exact Legacy v1 without winning the overlap comparison. Its paired mean-response ratio is 0.9370 (95% bootstrap interval [0.9303, 0.9433]), with 474 wins, 351 ties, and 175 losses. It raises deadline hit from 46.67% to 53.33%, reduces median makespan from 67.084 s to 62.421 s, and reduces median energy from 3828.224 J to 3738.191 J. Per-trace leaders remain different: EFT leads the data-locality wave, Adaptive leads mixed arrivals and priority-burst response, and historical Legacy leads the recovery/gang response comparison.

The seven-policy overlap ranking is likewise unchanged between the exploratory and confirmatory seed blocks. This stability coexists with trace-specific leaders and therefore supports objective-specific, not universal, recommendations.

7.4 Same-host Kubernetes results

The strict analyzer accepts all three topology documents. The matrix contains 27 trials, 135 completed workflows, and 540 successful final task records. Every policy completes 45/45 workflows and 180/180 final tasks. There are no failed final tasks, scheduler error lines, collection errors, or container restarts; every trial contains at least twenty binding records, the two exact Legacy vectors match their registered weights, and profile paths are isolated by policy. Table 7.5 reports descriptive timing by topology.

Topology	Policy	Median workflow (s)	P95 workflow (s)	Critical (s)	Makespan (s)	Preemptions
Balanced	Legacy v1	52.863	73.859	30.618	77.948	0
	Legacy tuned	50.427	71.121	31.558	73.933	0
	Adaptive	44.372	90.339	29.243	88.361	14
Homogeneous	Legacy v1	47.527	53.517	28.690	59.189	0
	Legacy tuned	47.527	54.558	28.187	57.083	0
	Adaptive	34.203	67.047	27.454	65.701	15
Skewed	Legacy v1	47.443	60.388	32.254	61.560	0
	Legacy tuned	42.992	51.401	23.558	57.382	0
	Adaptive	33.978	67.579	32.115	65.734	15

Table 7.5: Three sequential repetitions per policy and logical `kind` capacity profile. Every row has 15 completed workflows. The shared physical host and small n preclude population inference.

Tuned Legacy has a lower median makespan than v1 in all three topologies: 3.56% on homogeneous, 6.79% on skewed, and 5.15% on balanced. Across all nine trials per policy, tuned Legacy reduces v1 makespan by 3.50%, critical response by 5.47%, P95 workflow response by 1.60%, and median workflow response by 0.29%. The direction is not uniform for every metric: tuned ties homogeneous median response, worsens homogeneous P95 by 1.94%, and worsens balanced critical response by 3.07%. The live evidence nevertheless agrees with the simulation prediction that the selected vector improves the exact original scorer in aggregate.

Adaptive has the lowest median workflow response in every topology, beating tuned Legacy by 12.01% on balanced, 28.04% on homogeneous, and 20.97% on skewed. The aggregate improvement is 19.40%. This gain does not extend to every objective: relative to tuned Legacy, Adaptive raises aggregate makespan by 14.80%, P95 response by 1.69%, and critical response by 1.52%. Its makespan is higher in all three topologies. The shape medians in Table 7.6 identify where the trade is made.

Workflow shape	**Legacy v1 (s)**	**Legacy tuned (s)**	**Adaptive (s)**
Pressure	50.994	47.527	67.047
Critical	29.431	27.821	28.245
Pipeline	52.863	50.419	38.173
Fan-out	50.828	50.382	43.634
Parallel	41.121	44.240	29.871

Table 7.6: Aggregate median live workflow response by shape, nine observations per policy and shape across the three logical profiles.

Adaptive is fastest for pipeline, fan-out, and parallel workflows; tuned Legacy is fastest for pressure and, narrowly, critical workflows. The scheduler logs identify the mechanism directly: all 44 Adaptive victims are `pressure-long-*` Pods, while both Legacy variants perform no preemption. Deletion frees logical capacity for later work, but recreation discards pressure-task progress. This explains the faster regular-workflow median together with the 41.07% slower pressure median relative to tuned Legacy and the longer trial makespan.

The data and operational paths also execute successfully. Adaptive records five remote fetches and four local reads; tuned Legacy records four remote and five local; v1 records eight remote and one local. Observed transfer time totals 4.487 s, 5.453 s, and 11.354 s, respectively. These counts verify provenance, fileserver access, task-side fetch, and metric collection, but the same-host links do not support a physical network-quality claim. Kill/restart preemption uses Pod deletion and recreation, so zero final container restarts does not mean no work was discarded.

Profile accumulation does not produce monotonic wall-clock improvement. Across-topology median tuned-Legacy makespan changes from 58.401 to 57.292 to 59.440 s over repetitions one to three; v1 changes from 61.560 to 64.453 to 59.863 s. Mean response is also non-monotonic for both variants. Profiles alter later decisions, but cyclic position, image/cache state, interference, and host noise are concurrent changes. The retained sequence therefore rejects a guarantee that Legacy gets faster on every learned run; it does not show that learning is inert.

7.5 Cross-surface interpretation

The two surfaces agree on three qualitative points. First, informed placement is far safer than Random in heterogeneous scenarios. Second, no policy dominates all objectives and families. Third, prioritising urgent response through preemption can transfer delay to lower-priority work. They also agree on the tuned-v1 direction: tuned Legacy improves v1 in paired simulation, and lowers live makespan in all three logical profiles.

They do not agree numerically, nor should they. In overlapping simulation, Adaptive's median makespan is 11.78% below tuned Legacy; in live `kind`, it is 14.80% above tuned Legacy. Simulator Adaptive has full DAG knowledge, fixed scenario parameters, and no UCB learning, while live Adaptive sees the ready frontier, learns profiles, and restarts deleted Pods. All 44 live victims are pressure tasks, and the workers contend on one physical host. The reversal is therefore a boundary result: simulation predicts policy tendencies under its model, while `kind` reveals control-plane, interference, and restart costs. It is not a failed attempt at cycle-accurate validation.

7.6 Answers to the research questions

RQ1. For static closed DAGs, HEFT has the lowest mean makespan regret (1.69%) and P95 regret (5.75%); EFT is the close second. Family results still require specialists, so there is no universal winner.

RQ2. Under overlap, historical Legacy gives the best deadline hit rate (60.00%), Adaptive gives the best P95 response regret (35.07%), four policies share the lowest median makespan (55.070 s), and EFT/HEFT use the least median energy. Adaptive preemption improves the response tail but not median makespan over its ablation.

RQ3. Seeded optimization improves exact Legacy v1. In confirmation, tuned/v1 paired mean ratios are 0.9714 for closed makespan and 0.9370 for overlap response, with intervals wholly below one. Broad ablation identifies type affinity, locality, and load balance as consequential in this corpus; controlled cases verify that infrequent warm-image, failure, and memory factors are still active.

RQ4. Across three recreated capacity profiles, the live stack completes all 135 workflows and 540 final tasks, verifies exact weights and policy-isolated profiles, performs 44 real victim deletions/retries, and records 17 remote fetches plus ten local reads without task failure, restart, collection error, or scheduler error.

RQ5. Simulation and live evidence agree on objective tradeoffs, the qualitative role of preemption, and aggregate tuned-v1 improvement. Live kill/restart overhead reverses the Adaptive-versus-tuned makespan ordering. Cross-surface validation is therefore mechanism-aware, not numerical equivalence.

Chapter 8

Conclusion and further work

8.1 Summary

TaskScheduler answers a practical question rather than proving a universal scheduler: how should heterogeneous DAG placement be studied when task criticality, locality, contention, online uncertainty, thermal/failure risk, deadlines, and preemption interact?

The project contributes two working surfaces. A deterministic discrete-event simulator supports nine policies and explicit resource, network, thermal, energy, failure, checkpoint, gang, release, and preemption behaviour. A Kubernetes-native implementation supplies two CRDs, a dependency controller, exact Legacy v1, tuned Legacy, and Adaptive policies, persistent profiles, producer-local data, telemetry, binding, and kill/restart preemption. Their semantic boundary is documented rather than hidden: only the simulator has the complete DAG and its Adaptive policy does not perform live UCB learning.

The 17,400 exploratory and 33,250 fresh-seed confirmatory matrix records reject a single-winner conclusion. For closed DAGs, HEFT has the lowest confirmatory mean regret (1.69%) and P95 regret (5.75%); EFT is a close second. Under overlap, historical Legacy has the highest deadline hit rate (60.00%), Adaptive has the lowest P95 response regret (35.07%), four policies share the lowest median makespan (55.070 s), and EFT/HEFT have the lowest median energy. Tuned exact Legacy improves exact v1 pairwise on both surfaces but does not become the global winner.

The 27 same-host `kind` trials validate the complete control path across three logical capacity profiles. All 135 workflows and 540 final tasks succeed. Tuned Legacy lowers aggregate v1 makespan by 3.50% and does so in all three profiles. Adaptive lowers median workflow response by 19.40% relative to tuned Legacy, but raises makespan by 14.80% and issues 44 kill/restart preemptions; both exact Legacy variants issue none. Seventeen remote fetches and ten local reads validate both data paths. These

are descriptive integration results on one physical host, not physical multi-machine validation.

8.2 Policy recommendation

The appropriate policy follows the deployment objective:

Static, known DAGs.
Use HEFT as the default when full graph information is available. EFT is nearly equivalent in the global closed ranking and is simpler when critical-path ordering adds little.

Energy-sensitive overlap.
Prefer EFT (or HEFT when rank is useful). Both have 17.60% lower median simulated energy than Legacy and Adaptive while sharing the best median workload makespan.

Deadline and batch completion.
Prefer Legacy when deadline hit rate and avoiding discarded work dominate, but preserve the identity distinction: the deadline leader is historical simulator Legacy and is not the live eight-factor scorer. When exact live Legacy semantics are required, use tuned Legacy rather than v1; it improves v1 in both fresh-seed matrices and lowers live makespan in all three logical profiles.

Regular workflow response.
Use Adaptive when reducing median, fan-out, pipeline, and parallel-workflow response justifies kill/restart cost. Tuned Legacy is slightly better on aggregate critical response and substantially better on pressure work, while Adaptive has a worse P95 and total makespan. Preemption budgets must therefore be treated as product policy, not an invisible optimisation.

Table 8.1 translates family-level confirmation into workload guidance. Ties are preserved rather than broken arbitrarily, and historical Legacy remains labelled as a simulator-only identity.

The immediate engineering recommendation is not a more complicated score. It is a live Adaptive mode without preemption, or a stricter victim-benefit test that predicts urgent-task gain against discarded runtime and restart delay. The simulator ablation preserves the best median makespan with only a modest response-tail penalty and no interruptions, making this the most evidence-supported next experiment. It remains a hypothesis until tested through the Kubernetes path.

Workflow or trace type	**Confirmatory leader(s)**	**Deployment interpretation**
Cache-sensitive closed DAG	HEFT	Use full-DAG rank when the graph is known; this is the clearest single-family HEFT result.
CPU-heavy closed DAG	EFT, HEFT	EFT is the simpler choice when their critical-path ordering produces the same outcome.
Fan-out or mixed closed DAG	Adaptive, Adaptive no-preempt	Prefer no-preempt when restart cost dominates; placement/order, not interruption, drives the closed-DAG tie.
Diamond closed DAG	Adaptive pair, tuned Legacy	Tuned exact Legacy is a viable live-semantics choice; Adaptive remains suitable when priority handling is required.
Data-heavy, deep, linear, or wide DAG	Broad informed-policy ties	Start with EFT; use tuned Legacy only when exact live score compatibility is a requirement.
Data-locality wave	EFT response; EFT/HEFT makespan	Prefer EFT unless full-DAG rank has an independent operational use.
Mixed arrivals	Adaptive response; Adaptive pair makespan	Adaptive is the response choice; use the no-preempt variant when interruption cost is high.
Priority burst	Adaptive response; tuned Legacy deadline	Select by service objective rather than one aggregate rank.
Recovery/gang mix	Historical Legacy response; Legacy v1 deadline/makespan	Treat this as scenario-specific evidence; the response leader is not the live Legacy implementation.

Table 8.1: Workflow-specific guidance from fresh-seed simulation. These recommendations do not substitute for the same-host live shape results or physical-cluster validation.

8.3 Limitations

The simulation matrix is broad but inherits its calibrated/synthetic scenario assumptions. Bootstrap intervals measure variation across the designed corpus rather than an external cluster population. The live study has three sequential repetitions per policy, evolving profiles, synthetic load-derived thermal signals when sysfs is unavailable, and worker containers that share one Windows/Docker host. Profile sequences are non-monotonic and confounded by cyclic position, image/cache state, and host load. The study validates software mechanisms but not physical network isolation, real thermal throttling, hardware energy, or node failure recovery.

The live scheduler receives only ready Pods and currently reconstructs an empty future-edge list, so no claim about live HEFT upward ranking is supported. The compared live policies are exact Legacy v1, tuned exact Legacy, and Adaptive; the Kubernetes default scheduler is not a baseline in the definitive artifact. Physical `k3s` scripts and multi-architecture build paths are implementation assets, not completed experimental evidence.

8.4 Further work

1. Repeat the balanced policy experiment on at least three independent physical machines, with controlled cross-over order, cold and warm profile phases, real link constraints, and hardware thermal/energy collection. This is the required external-validation step.

2. Pass the complete Workflow DAG or controller-computed ranks to the scheduler, then compare live EFT, HEFT, Legacy, Adaptive, and Adaptive without preemption on identical ready frontiers.

3. Replace threshold-only victim selection with an explicit benefit inequality that includes victim progress, restart/startup cost, remaining urgent work, and node availability. Add checkpoint handoff where the task runtime supports it.

4. Run live mechanism ablations for profile learning, UCB, locality, thermal penalty, failure penalty, and preemption. The exact Legacy pair isolates weights, but the Adaptive/Legacy contrast changes several mechanisms at once and cannot assign causal effect to each score term.

5. Calibrate simulator uncertainty from repeated physical microbenchmarks, then compare predicted task/transfer distributions with live observations rather than comparing only policy order. Add scheduling latency, isolated slowdown, fairness, and continuous thermal exposure only after the harness records them directly.

8.5 Closing remarks

TaskScheduler demonstrates that workflow-aware heterogeneous scheduling can be carried from explicit models to a functioning Kubernetes control plane, and that doing so changes the answer. The additional mechanisms do not produce a universally superior policy. They expose a decision: whether energy, deadlines, batch completion, or urgent response matters most, and how much interrupted work the system is permitted to spend on that objective. Keeping that decision visible is the project's most defensible conclusion.

Bibliography

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, *Performance-effective and low-complexity task scheduling for heterogeneous computing*, IEEE Trans. on Parallel and Distributed Systems, 13(3):260–274, 2002.

[2] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, in Proc. AFIPS Spring Joint Computer Conference, pp. 483–485, 1967. doi: 10.1145/1465482.1465560.

[3] S. Williams, A. Waterman, and D. Patterson, *Roofline: an insightful visual performance model for multicore architectures*, Communications of the ACM, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[4] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talwar, and A. Goldberg, *Quincy: fair scheduling for distributed computing clusters*, in Proc. 22nd ACM Symposium on Operating Systems Principles, pp. 261–276, 2009. doi: 10.1145/1629575.1629601.

[5] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, *Dominant Resource Fairness: Fair Allocation of Multiple Resource Types*, in Proc. 8th USENIX Symposium on Networked Systems Design and Implementation, pp. 323–336, 2011.

[6] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, *Omega: flexible, scalable schedulers for large compute clusters*, in Proc. 8th ACM European Conference on Computer Systems, pp. 351–364, 2013. doi: 10.1145/2465351.2465386.

[7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, *Large-scale cluster management at Google with Borg*, in Proc. 10th European Conference on Computer Systems, pp. 1–17, 2015. doi: 10.1145/2741948.2741964.

[8] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, *Sparrow: distributed, low latency scheduling*, in Proc. 24th ACM Symposium on Operating Systems Principles, pp. 69–84, 2013. doi: 10.1145/2517349.2522716.

[9] J. Moore, J. Chase, P. Ranganathan, and R. Sharma, *Making Scheduling "Cool": Temperature-Aware Workload Placement in Data Centers*, in Proc. USENIX Annual Technical Conference, 2005.

[10] The Kubernetes Authors, *Pod Priority and Preemption*, https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/, accessed August 2026.

[11] The Kubernetes Authors, *Custom Resources*, https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/, accessed August 2026.

[12] The Kubernetes Authors, *DaemonSet*, https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/, accessed August 2026.

[13] Kubernetes SIG Testing, *kind Quick Start*, https://kind.sigs.k8s.io/docs/user/quick-start/, accessed August 2026.

[14] P. Auer, N. Cesa-Bianchi, and P. Fischer, *Finite-time analysis of the multiarmed bandit problem*, Machine Learning, 47(2–3):235–256, 2002.

[15] The Kubernetes Authors, *Kubernetes documentation*, https://kubernetes.io/docs/, accessed 2026.

[16] The Kubernetes Authors, *Kubernetes Scheduler*, https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/, accessed August 2026.

[17] Rancher Labs, *k3s: Lightweight Kubernetes*, https://k3s.io/, accessed 2026.